ErrorBuddy

A beginner compiler that explains its own errors: the right line, plain words, one concrete fix.

Compiler Design Project · C++17

Architecture, every phase, evaluation and scope in one deck

```
$ ./errorbuddy
missing_semicolon.txt

Error on line 2:
Expected ';' at the end of the
statement.
Suggestion: Add a semicolon
after '10'.

(Found during: Syntax Analysis)
```

ErrorBuddy at a glance

5	3	~730
compiler phases	C++ source files	lines of C++

10/10	24/24
tests pass (make test)	mistakes on the fix line

IN THIS DECK

01 **Problem and idea**
why compiler errors confuse beginners

02 **The language**
what ErrorBuddy compiles

03 **Architecture**
files, data flow, one compile end to end

04 **The five phases**
lexer, parser, symbol table, TAC, errors

05 **Tools and evaluation**
playground, tests, case study

06 **Scope**
built today versus planned

01

Problem and idea

Compiler errors are written for people who already know the grammar.

Compiler errors talk like grammars

PROGRAM (EQUIVALENT C)

```
1  int a = 5;
2  int b = 10          <- real mistake
3  printf("%d", a + b);
```

GCC 11.4 SAYS

```
program.c:3:5: error:
expected ',' or ';'
before 'printf'
```

Wrong line

Points at line 3. The fix belongs on line 2.

Jargon

"expected ... before" is grammar talk, not plain English.

No fix

Says what the parser wanted, never what to type.

Every error is a structured object

```
Error on line 2:
Expected ';' at the end of the statement.
Suggestion: Add a semicolon after '10'.

(Found during: Syntax Analysis)
```

Same mistake, ErrorBuddy output

```
struct CompileError {
    int    line;
    string phase;
    string message;
    string suggestion;
};
```

LINE 2

where the fix belongs, not where parsing stopped

PHASE Syntax Analysis

lexical, syntax or semantic

MESSAGE Expected ';' at the end…

what rule was broken, in plain English

SUGGESTION Add a semicolon…

one concrete thing to type

Four objectives

1

Build a working compiler

Source text in, three-address code (TAC) out.

2

Cover every core phase

Lexical, syntax, semantic, intermediate code and error reporting.

3

Report errors like a tutor

The right line, plain words, one concrete fix.

4

Hand-written C++17 only

Standard library, no Lex or Yacc, so each message is ours.

02

The language

A small, C-like beginner language, kept small on purpose so every phase is complete.

What ErrorBuddy compiles

EXAMPLES/VALID.TXT

```
// A valid ErrorBuddy program
int age = 20;
int score = 10;
score = score + age * 2;
int bonus = (score - 5) / 5;
print score;
print bonus * (age + 1);
```

GRAMMAR (ONE C++ FUNCTION PER RULE)

```
program     → statement*
statement   → declaration
            | assignment | print
declaration → int ID = expr ;
assignment  → ID = expr ;
print       → print expr ;
expression  → term ((+|-) term)*
term        → factor ((*|/) factor)*
factor      → NUM | ID | ( expr )
```

Has

int variables with a starting value, + - * /, ( ), print, // comments

Leaves out

unary minus, if/else, loops, functions, input

Stops

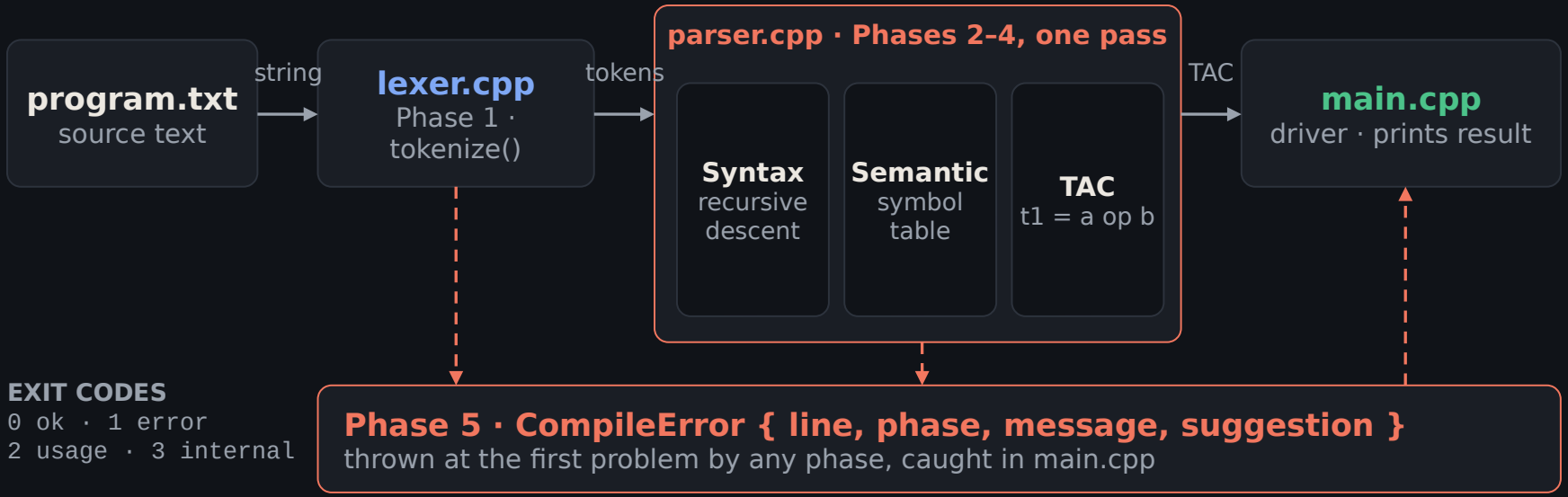
at the first error, so no partial TAC is shown

03

Architecture

Five phases in three C++ files, one error channel, three ways to run it.

Five phases, three source files



WAYS TO RUN THE SAME BINARY

Terminal
`./errorbuddy file.txt [--tokens]`

make test
`diff against examples/expected`

Web playground
`web/server.py → JSON → browser`

What each file does

FILE	LINES	ROLE
`main.cpp`	88	Driver: reads the file, runs the phases, prints TAC, symbols and phase summary or the error; sets the exit code
`lexer.h / lexer.cpp`	48 / 114	Phase 1: Token, TokenType, tokenize(). Also defines CompileError
`parser.h / parser.cpp`	80 / 396	Phases 2–4 in one pass + most of Phase 5 (diagnosis helpers, typo hints)
`Makefile`	45	make, run, test, web, clean · g++ -std=c++17 -Wall -Wextra -pedantic -O2
`examples/`	10 + 10	Test programs and their expected output in examples/expected/*.out
`web/`	89 / 288	server.py + index.html: browser playground that runs the real binary
`case_study/`	972	24-mistake corpus, ErrorBuddy vs gcc study, hint and lexer experiments

One compile, end to end

1

Start

main() reads path + --tokens, loads the whole file into one string

→

2

Lex

tokenize() → vector<Token>, each with type, text and line

→

3

Parse + emit

for every statement: grammar, symbol table, append TAC

→

4

Finish

main() prints the result and sets the exit code

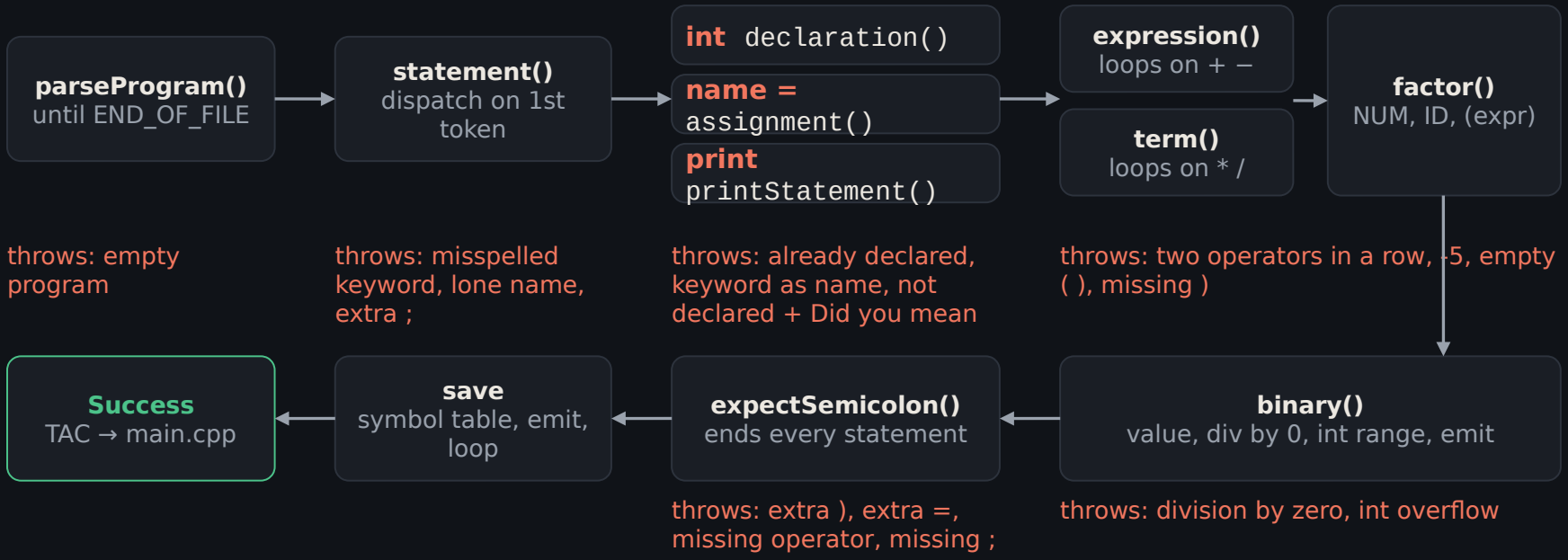
EVERY PHASE PASSED · EXIT 0

```
Compilation successful!
    t1 = b * 2
    t2 = a + t1
    c = t2
[OK] 1. Lexical Analysis - 22 tokens …
```

FIRST PROBLEM FOUND · EXIT 1

```
Error on line 3:
Division by zero. Variable 'zero'
is 0 at this point …
Suggestion: Make sure the value
on the right of '/' is not zero.
```

Inside the parser



After save, statement() runs again until END_OF_FILE. Every throw is a CompileError caught by main.cpp.

04

The five phases

Lexer, parser, symbol table, three-address code, and the error engine.

Phase 1 · The lexer makes tokens

INPUT

```
int c = a + b * 2;
```

OUTPUT: VECTOR<TOKEN> (TEXT · TYPE · LINE)

int	c	=	a	+	b	*	2	;
KEYWORD	IDENT	ASSIGN	IDENT	OPERATOR	IDENT	OPERATOR	NUMBER	SEMI

Skips and tracks

- spaces and // comments
- counts newlines, so every token carries its line

Rejects (lexical errors)

- unknown symbols such as @
- numbers above INT_MAX
- names that start with a digit: 2nd

Hands on

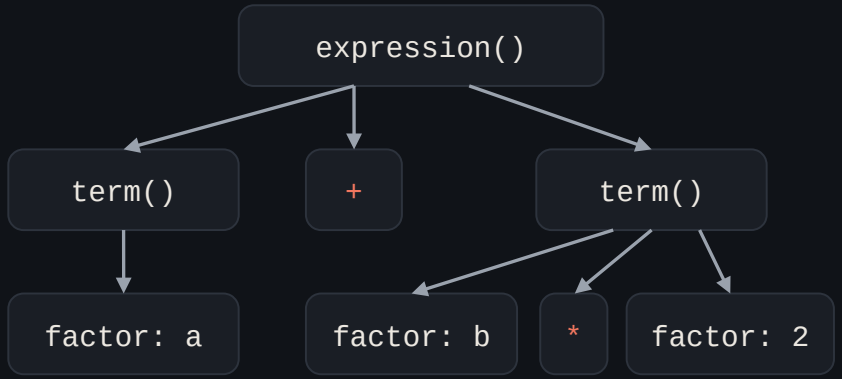
- token list ending in END_OF_FILE
- --tokens prints it as a line / type / text table

Phase 2 · Recursive descent

C++ SHAPE OF THE RULES

```
Operand expression() {
  Operand l = term();
  while (check(PLUS) || check(MINUS))
    l = binary(l, advance(), term());
  return l;
}
Operand term() {    // same, * and /
  ... factor() ...
}
```

CALL TREE FOR A + B * 2



b * 2 finishes inside the right-hand term() before + is applied, so the TAC computes it first.

Precedence comes from the grammar, not a table. Loops make operators left-associative: 10 - 4 - 3 = (10 - 4) - 3.

Parse, check and emit in one pass

1. **`declaration()`**
   sees int c =, calls expression()

2. **`term()` → `a`**
   first operand, + is next

3. **`term()` → `b * 2`**
   binary() emits t1 = b * 2

4. **`expression()`**
   binary() emits t2 = a + t1

5. **`expectSemicolon()`**
   ; found, c enters the symbol table, emit c = t2

TAC · PRECEDENCE.TXT

```
a = 5
b = 10
t1 = b * 2
t2 = a + t1
c = t2
print c
```

This is syntax-directed translation: syntax, semantics and code generation share one walk over the tokens.

Phase 3 · Symbol table checks

SYMBOL TABLE AFTER VALID.TXT

NAME	DECLARED	VALUE
age	line 2	20
score	line 3	50
bonus	line 5	9

Why values are tracked
No input and no loops means every value is known while compiling, so division by a variable that is 0 is caught before the program runs.

Undeclared or misspelled
`scroe = 2;` → Did you mean 'score'?

Declared twice
'age' already declared on line 1

No starting value
`int a;` → needs '= value'

Used in its own declaration
`int a = a + 1;`

Division by zero · overflow
`a / zero`, `8 / (4 - 4)`, int range

Phase 4 · Three-address code

SOURCE

```
int age = 20;
int score = 10;
score = score + age * 2;
int bonus = (score - 5) / 5;
print score;
print bonus * (age + 1);
```

TAC (12 INSTRUCTIONS)

```
age = 20
score = 10
t1 = age * 2
t2 = score + t1
score = t2
t3 = score - 5
t4 = t3 / 5
bonus = t4
print score
t5 = age + 1
t6 = bonus * t5
print t6
```

One operator
per instruction: t1 = a op b

Safe temps
t1, t2… skip names already used

All or nothing
printed only if every phase passes

Phase 5 · Errors that teach

MISTAKE	PHASE	ERRORBUDDY SAYS
`int b = (a + 2)) * 3;`	Syntax	Found an extra ')' that has no matching '('.
`int y = (x + 3 * 4;`	Syntax	Missing ')' to close the '(' opened on line 2. Add ')' after '4'.
`int y = x + * 3;`	Syntax	Two operators in a row: '+' followed by '*'.
`int a = -5;`	Syntax	Negative numbers such as '-5' are not supported directly. Write 0 - 5.
`prnt x;`	Syntax	Unknown word 'prnt'. Did you mean the keyword 'print'?
`total + marks`	Semantic	Variable 'marks' has not been declared. Declare it: 'int marks = 0;'
`int age = 19;`	Semantic	Variable 'age' has already been declared (on line 1).
`int x = 2nd;`	Lexical	'2nd' is not a valid name. Variable names cannot start with a digit.

Diagnosis reads nearby tokens

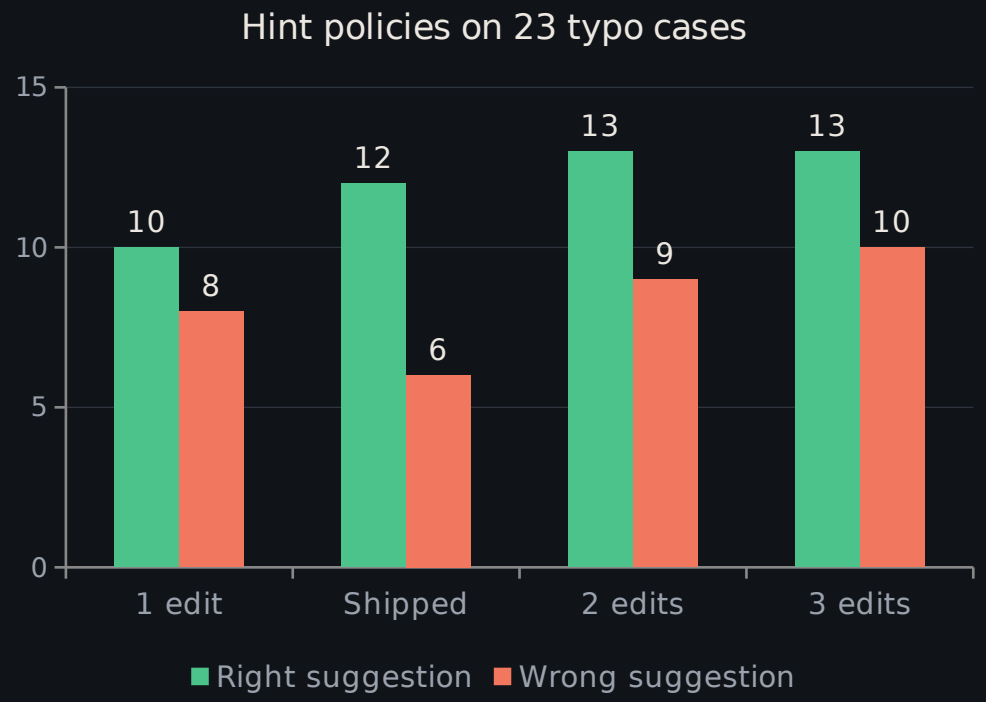
`expectSemicolon()` the statement is over, but the next token is not ';'. What is it?

`)`	→ Found an extra ')' that has no matching '('.
`=`	→ Unexpected '='. A statement can contain only one '='.
`x 5 ( same line`	→ Missing operator between the two values.
`anything else`	→ Expected ';', reported on the line of the previous token.

`factorError()`
a value was expected: two operators in a row, -5, empty ( ), keyword inside an expression, program ended mid-expression

`unknownStatementError()`
a statement starts wrongly: misspelled or capitalised keyword, a name on its own, an extra ';', '=' with no name

Typo hints use edit distance



EXAMPLES

```
scroe → score   2 edits
prnt  → print   1 edit
mx    → max     1 edit
```

Shipped rule

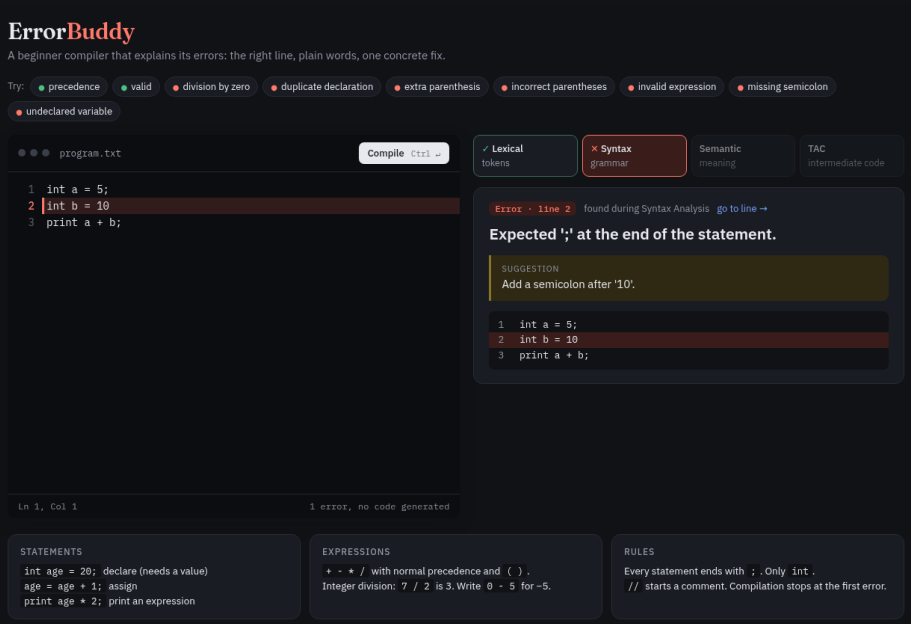
1 edit for names of up to 3 letters, 2 for longer names. Never suggest the word itself. Fewest wrong hints (6) with 12 right.

05

Tools and evaluation

A browser playground, an automated test suite, and a case study against gcc.

Seeing it work: the web playground



`make web` → `http://localhost:8795`

Editor
recompiles ~350 ms after typing stops

↓

POST /api/compile
temp file → ./errorbuddy --tokens, 5 s timeout

↓

server.py
regex turns stdout into JSON: tokens, error, TAC, symbols

↓

Page
highlights the error line, lights the phase strip

Testing: make test

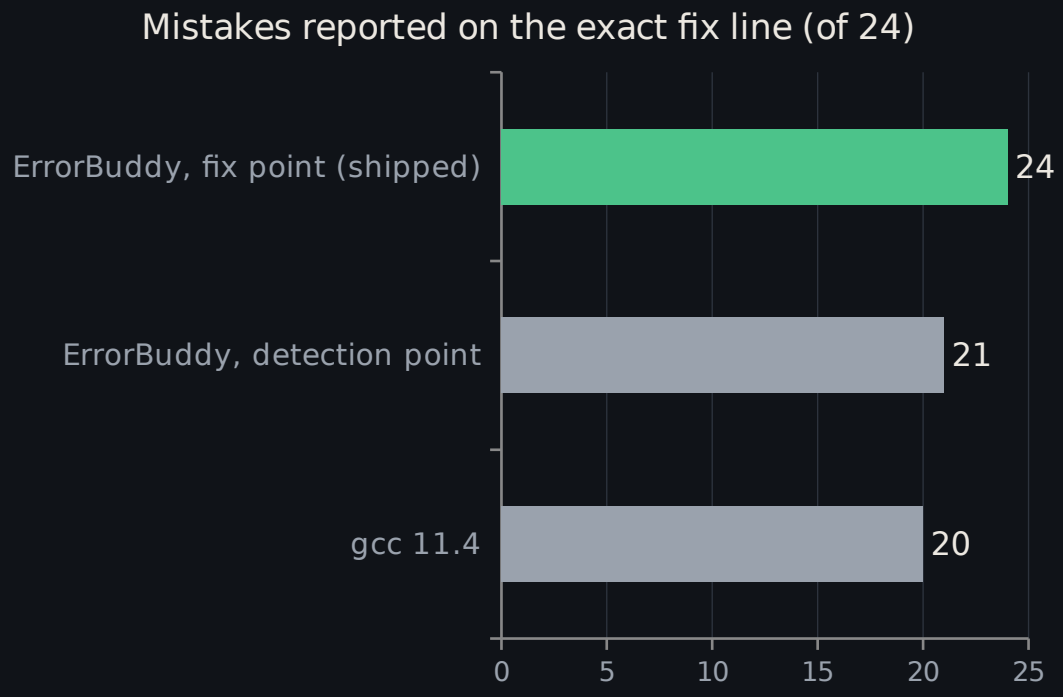
PASS	`valid` full program, 12 TAC instructions	PASS	`precedence` b * 2 computed before +
PASS	`missing_semicolon` reported on line 2, not 3	PASS	`undeclared_variable` 'marks' has not been declared
PASS	`duplicate_declaration` 'age' already declared on line 1	PASS	`invalid_expression` two operators in a row
PASS	`incorrect_parentheses` missing ')' for '(' on line 2	PASS	`extra_parenthesis` extra ')' with no '('
PASS	`division_by_zero` variable 'zero' is 0	PASS	`empty` nothing to compile

10/10

example programs match their expected output

```
g++ -std=c++17
 -Wall -Wextra
 -pedantic -O2
```

Case study: reporting the fix line



24/24 mistakes caught by ErrorBuddy (gcc: 23)

0 reports later than the fix line (gcc: 3)

3 late gcc cases, all a missing ';'

Lexer rules, tested by removal

// comments after '/'

```
int a = 10 / 2; // half
```

SHIPPED

OK

RULE REMOVED

A statement cannot start with '/'.

No leading digit

```
int 2nd = 5;
```

SHIPPED

'2nd' is not a valid name. Names cannot start with a digit.

RULE REMOVED

Expected a variable name after 'int', but found '2'.

Whole-word keywords

```
int interest = 5;
```

SHIPPED

OK

RULE REMOVED

'int' is a reserved keyword and cannot be used as a variable name.

06

Scope and conclusion

What is built and tested today, and what the original proposal planned next.

Built today, and what comes next

BUILT AND TESTED

- int variables with a required starting value
- + - * / with precedence and parentheses
- print any expression; // comments
- TAC, symbol table and phase summary
- Four-field errors with line-accurate hints
- Web playground and make test (10/10)

NEXT, FROM THE PROPOSAL (NOT BUILT)

- if / else, while and for loops
- Comparison and logical operators, printf
- Error recovery: several errors per run
- Functions and arrays; char and float
- Constant folding and other optimisations
- Assembly backend from TAC

ErrorBuddy

A complete small compiler, lexer to three-address code, whose main contribution is errors that teach: the right line, plain words, and a concrete fix.

Thank you

TRY IT

```
$ make test
$ make run FILE=examples/valid.txt
$ make web
  → http://localhost:8795
$ python3 case_study/run_study.py
```